

Sors ton notebook du .ipynb, déploie une app Streamlit



8 étapes pour transformer une analyse Jupyter en app en ligne, avec un lien qu'un recruteur peut ouvrir.

Un notebook prouve que tu sais analyser. Il ne prouve pas que tu sais livrer. Le nombre de bons notebooks qui finissent oubliés dans un dossier Downloads est la vraie raison de ce guide : ce fil te fait passer de l'analyse locale à une app déployée, testée par n'importe qui, sans installation.

Sortir le code du notebook.

Un notebook mélange exploration et logique utile. Une app ne garde que la logique utile.

1. Isole ton code utile dans un fichier `app.py`

```
import pandas as pd

def charger_ventes():
    df = pd.read_csv("data/ventes.csv")
    return df

def calculer_ca_par_mois(df):
    return df.groupby("mois")["montant"].sum()
```

Piège fréquent : garder les `print()` et les cellules d'exploration du notebook (`df.head()`, essais ratés, tests) dans `app.py`. Un `print()` ne s'affiche nulle part dans Streamlit, et une cellule d'essai encore présente peut planter l'app entière au démarrage.

Résultat visible : un fichier `app.py` qui s'exécute avec `python app.py` sans erreur, avec seulement des fonctions, aucune cellule de brouillon.

2. Installe Streamlit et écris ta première page

```
pip install streamlit
```

`app.py`

```
import streamlit as st

st.title("Ventes 2025, vue rapide")
st.write("Chiffre d'affaires par mois, filtrable par region.")
```

Piège fréquent : lancer directement `python app.py`. Streamlit n'est pas un script Python classique : il faut le lancer via `streamlit run app.py`, sinon rien ne s'affiche et aucune erreur ne l'explique clairement.

Résultat visible : un onglet de navigateur s'ouvre sur `localhost:8501` avec ton titre et ta phrase d'intro, sans rien coder en HTML.

Construire l'app qui montre quelque chose.

Ici, tu passes d'une page vide à un vrai écran de travail : données, tableau, graphique, filtre.

3. Charge tes données avec `st.cache_data`

```
import streamlit as st
import pandas as pd

@st.cache_data
def charger_ventes():
    return pd.read_csv("data/ventes.csv")

df = charger_ventes()
```

Piège fréquent : utiliser un chemin absolu type `C:/Users/toi/Desktop/ventes.csv`. Ça marche en local et plante au déploiement : le serveur cloud n'a pas ton disque dur. Utilise un chemin relatif au repo, `data/ventes.csv`, avec le fichier committé dans le repo Git.

Résultat visible : le CSV se charge une seule fois grâce au cache `st.cache_data`, même si tu recharges la page vingt fois de suite.

4. Affiche un tableau et un graphique interactifs

```
st.subheader("Detail des ventes")
st.dataframe(df[["mois", "region", "montant"]])

st.subheader("Evolution du CA")
ca_mensuel = df.groupby("mois")["montant"].sum()
st.line_chart(ca_mensuel)
```

Piège fréquent : afficher le DataFrame brut avec toutes ses colonnes techniques (ids, timestamps, colonnes de calcul intermédiaire). Un recruteur qui ouvre ton lien voit un mur de chiffres, pas une analyse. Sélectionne les colonnes utiles avant l'affichage.

Résultat visible : un tableau lisible et une courbe qui montre la tendance du CA, sans scroll horizontal infini.

5. Ajoute un filtre qui recalcule avec `st.selectbox`

```
regions = df["region"].unique()
choix = st.selectbox("Region", regions)

df_filtre = df[df["region"] == choix]
st.line_chart(df_filtre.groupby("mois")["montant"].sum())
```

Piège fréquent : chercher à écrire un callback JavaScript ou une logique de rafraîchissement manuelle. Streamlit réexécute tout le script à chaque interaction : c'est justement ce comportement qui fait qu'un simple `st.selectbox` suffit à recalculer le graphique, sans code additionnel.

Résultat visible : tu changes la région dans le menu déroulant, le graphique se met à jour sous tes yeux, sans recharger la page.

À vérifier avant de committer

Avant de figer les dépendances et de pousser sur GitHub, relance l'app une dernière fois en local.

Démarrage

`streamlit run app.py` sans erreur ni warning bloquant.

Chemins

Que des chemins relatifs (`data/...`), aucun `C:/` ni `/home/...`

Filtre

Le `selectbox` recalcule bien le graphique à chaque changement.

Rendre l'app partageable.

Une app qui tourne sur ta machine n'existe pour personne d'autre. Ici, tu la fais exister ailleurs.

6. Fige tes dépendances : `requirements.txt` et un `README`

`requirements.txt`

```
streamlit==1.38.0
pandas==2.2.2
```

`README.md`

```
App ventes
Analyse du CA mensuel par region.
Lancer en local : streamlit run app.py
```

Piège fréquent : écrire `streamlit` sans numéro de version, ou copier tout ton environnement Anaconda dans `requirements.txt`. Un fichier trop large avec des dizaines de paquets ralentit le build et peut le faire échouer sur une dépendance système absente du cloud.

Résultat visible : un `requirements.txt` de quelques lignes qui installe exactement ce que l'app utilise, rien de plus, et un `README` qu'un recruteur lit en 10 secondes.

7. Pousse ton code sur [GitHub](#)

Piège fréquent : committer un notebook énorme rempli de sorties et d'images comme fichier principal du repo. Un recruteur ouvre le repo, voit un `.ipynb` de 40 Mo illisible sur GitHub, et referme l'onglet avant de trouver ton app.

Checklist avant de pousser

Un repo mal rangé fait fuir un recruteur avant même qu'il voie l'app.

Dépôt public

Pas privé, sinon Streamlit Cloud ne peut pas le lire sans configuration supplémentaire.

`app.py` à la racine

Pas caché dans un sous-dossier `notebooks/`.

`requirements.txt` à côté

Au même niveau que `app.py`, pas dans un sous-dossier.

Pas de notebook en vitrine

Le `.ipynb` reste une référence dans `notebooks/`, pas le fichier principal du repo.

Résultat visible : un repo minimal, lisible en une page GitHub : `app.py`, `requirements.txt`, `README.md`, `data/`.

8. Déploie sur [Streamlit Community Cloud](#) et récupère le lien public

Piège fréquent : déployer une fois puis ne jamais vérifier le lien. Une dépendance mal fixée ou un fichier de données oublié dans le repo peut faire planter l'app en silence après le premier déploiement : le lien affiche une erreur au premier visiteur, pas à toi.

Checklist de déploiement

Streamlit Community Cloud propose un déploiement gratuit depuis un repo GitHub public : connecte, choisis, déploie.

Connecter

Compte GitHub relié sur `share.streamlit.io`.

Configurer

Repo, branche, et `app.py` comme fichier d'entrée.

Tester

Le lien public s'ouvre en navigation privée, sans ton compte connecté.

Résultat visible : un lien du type `https://tonapp.streamlit.app` que tu peux coller dans un CV, un profil LinkedIn ou un message, et qui ouvre l'app directement, sans installation côté visiteur.

8 sur 8. Tu n'as plus un notebook local, tu as une app en ligne. C'est ce lien, pas un screenshot dans un PDF, qui montre que tu sais livrer, pas seulement analyser. Colle-le dans ton CV, dans ton profil LinkedIn, dans ta candidature : un recruteur qui clique et qui voit l'app tourner retient ton nom plus vite qu'une capture d'écran.

Un lien vaut mieux qu'un screenshot.

Si ce guide t'a aidé à sortir ton premier notebook du dossier Downloads, partage-le : il peut débloquent quelqu'un d'autre. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire, c'est là que je pioche mes prochains guides.

Dylan

